Project: Milestone 3 Deliverables and Checklist

Hardware for Artificial Intelligence and Machine Learning (HW4AI)

ECE 410/510

Spring 2026

Goal of Milestone 3

Milestone 3 is the integration and synthesis milestone. The compute core and the interface module verified separately in M2 must now talk to each other end to end, and the integrated design must be pushed through OpenLane 2. By May 24, a co-simulation must demonstrate data flowing from the host through your interface into the compute engine and results returning to the host, and you must have a synthesis result, a documented synthesis failure with a revised scope, or a documented scope adjustment with the synthesis attempt that justifies it.

M3 is the last milestone where revision tokens are accepted. M4 has no revision. If you arrive at M4 having never run OpenLane on your design, the project is over. The single most common failure mode in this course is treating synthesis as something to do after the design is "done"; tools surface constraints you did not anticipate, and the only way to find them is to run the flow.

How to submit

All M3 deliverables are submitted by pushing to your public GitHub repository. There is no Canvas upload.

Expected repository structure for M3

Your repository should contain at minimum the following layout after M3. M1 and M2 paths must still be present. M3 adds `project/m3/`.

```
your-repo/
├── README.md
├── project/
│   ├── heilmeier.md
│   ├── m1/                                ← from M1
│   ├── m2/                                ← from M2
│   └── m3/
│       ├── README.md                      ← catalogs all M3 files
│       ├── rtl/
│       │   └── top.sv                     ← integrated top module
│       ├── tb/
│       │   └── tb_top.sv                  ← end-to-end co-sim testbench
│       ├── sim/
│       │   ├── cosim_run.log              ← co-sim transcript with PASS
│       │   └── cosim_waveform.png         ← end-to-end waveform
│       ├── synth/
│       │   ├── config.json                ← OpenLane 2 configuration
│       │   ├── openlane_run.log           ← full OpenLane stdout/stderr
│       │   ├── timing_report.txt          ← STA report
│       │   ├── area_report.txt            ← area / cell count
│       │   ├── power_report.txt           ← power estimate (if available)
│       │   └── critical_path.md           ← critical path identification
│       └── synthesis_notes.md             ← narrative: what worked, what did not
└── codefest/                              ← from earlier weeks
```

The `project/m3/` folder is new. The `project/m3/README.md` file is required and must catalog every file in the M3 folder with a one-line description of its contents. The grader reads this file before scoring; missing or incorrect entries make every other deliverable harder to verify.

Deliverable checklist

Check off each item only when the file is committed and pushed. A checked box that points to a missing or empty file is still Not Yet. The grader runs an automated check on every path; logs and reports must be committed so the grader can read them without re-running OpenLane.

Deliverable	GitHub file path
1. M3 folder README	
☐ README.md catalogs every file and subfolder in project/m3/ *One line per file: relative path and a brief description of contents. The grader uses this as the index to your M3 submission.*	`project/m3/README.md`
☐ README states the simulator used and how to reproduce the co-simulation *Exact command line. Tool versions. If preprocessing is required, list the dependencies.*	`project/m3/README.md`
☐ README states the OpenLane 2 version and how to reproduce the synthesis run *Commit SHA, tag, or release of OpenLane 2. Path to your configuration file. Any environment variables required.*	`project/m3/README.md`
2. Integrated top module	
☐ Top module instantiates the interface and the compute core and connects them *All inter-module signals named and wired. No floating ports. No stub modules: the actual M2 modules must be instantiated.*	`project/m3/rtl/top.sv`
☐ Port list documented in a header comment block *Each external port: name, direction, width, role. Same convention as M2.*	`project/m3/rtl/top.sv`
☐ Any glue logic between interface and compute core is identified and explained *FIFOs, handshake adapters, clock-domain crossings, width converters. If you needed glue, name it.*	`project/m3/rtl/top.sv`
3. End-to-end co-simulation	
☐ Testbench drives the interface from the host side and reads results back through the interface *No direct access to compute-core ports. The testbench must use the same protocol a real host would use.*	`project/m3/tb/tb_top.sv`

☐ Testbench uses an input that exercises the dominant kernel from M1 *The vector should match the kernel size you defended in M1 profiling. A 2x2 toy input for what was profiled as a 64x64 matmul is Not Yet.*	`project/m3/tb/tb_top.sv`
☐ Result returned through the interface is compared to an independent reference *Same rule as M2: the expected value must come from a software model or hand calculation, not from a prior DUT run.*	`project/m3/tb/tb_top.sv`
☐ Testbench prints PASS or FAIL based on the comparison *Single unambiguous PASS/FAIL line. The grader reads the log, not the waveform.*	`project/m3/tb/tb_top.sv`
☐ Co-simulation log committed showing PASS *Plain text including the PASS line. Must be the output of an actual simulation run, not a hand-edited file.*	`project/m3/sim/cosim_run.log`
☐ End-to-end waveform image committed *PNG or PDF showing host-side write transaction, internal compute activity, and host-side read of the result. Annotate the three regions.*	`project/m3/sim/cosim_waveform.png`

4. OpenLane 2 synthesis

☐ OpenLane 2 configuration committed *JSON or equivalent. Clock period, design name, source file list, any constraints you set.*	`project/m3/synth/config.json`
☐ Full OpenLane run log committed *Captured stdout and stderr from the OpenLane invocation. If it failed, the error must be in this log.*	`project/m3/synth/openlane_run.log`
☐ Timing report committed (if synthesis succeeded) *Worst negative slack, hold and setup checks, clock period actually achieved. If synthesis failed, this row is N/A; document the failure under section 5.*	`project/m3/synth/timing_report.txt`
☐ Area report committed (if synthesis succeeded) *Total cell area, cell count by type, and module-level breakdown if available.*	`project/m3/synth/area_report.txt`
☐ Critical path identified and explained *Name the start point, end point, and logic stages on the critical path. State why it is the critical path and what would shorten it. One paragraph minimum.*	`project/m3/synth/critical_path.md`
☐ Power estimation attempted; result or failure recorded *M4 requires a power estimate, so attempt the flow now. If power estimation succeeds, commit the report. If it fails, commit a short file documenting the failure mode and what you will try for M4.*	`project/m3/synth/power_report.txt`

5. Synthesis notes and scope status

☐ Narrative document explains what synthesized, what did not, and why	`project/m3/synthesis_notes.md`

One page minimum. Specific: which modules passed, which failed, what error messages appeared, what you changed to get further. "It worked" is not a narrative.

`project/m3/synthesis_notes.md`

☐ If scope was adjusted, the adjustment is stated explicitly with rationale

What was removed, what remains, what was substituted. Why the new scope is achievable. How M4 benchmarks will still be meaningful relative to your M1 baseline.

`project/m3/synthesis_notes.md`

☐ Document is ≥500 words

Below this length, the narrative almost certainly does not contain enough detail for the grader to assess scope adjustment.

Common reasons for Not Yet

The following are the most frequent reasons an M3 submission receives Not Yet. Review each before submitting.

Co-simulation bypasses the interface

If your testbench pokes the compute core directly instead of going through the interface, you have not demonstrated end-to-end data flow. The interface must be the only path between host and compute. This is the most common M3 failure mode.

Synthesis log committed but no reports

OpenLane produces multiple report files. Committing only the top-level log without timing and area reports is Not Yet. If those files do not exist because synthesis failed, document the failure in `synthesis_notes.md` and reference the error in the OpenLane log.

Critical path identified as "the longest one"

Naming the critical path means stating the start register, end register, and the logic stages between them. "The multiplier" is not a critical path; "the multiplier output feeding the accumulator through the saturation logic" is closer.

Power estimation skipped entirely

M4 requires a power estimate. M3 is when you find out whether the flow works. If you have not attempted power estimation by M3, M4 will be the first time, and that is the wrong place to discover the flow does not work on your design. Attempt it. Document what happens.

Scope adjustment without rationale

Adjusting scope is fine. Adjusting scope without explaining what changed, what remains, and why the remaining work still answers the M1 question is Not Yet. The grader cannot infer your reasoning from a smaller design.

README does not catalog the M3 folder

The `project/m3/README.md` file is the entry point for grading. If it does not list every file in the folder, or if it lists files that are not there, the grader has to guess what you intended. Guesses do not result in S.

Filenames or paths do not match the checklist

The grader matches paths exactly. `project/m3/Synth/` is not `project/m3/synth/`. Use lowercase. Match the names in this checklist character for character.